

MI BF API

Version 1.2

© 2021 SigmaStar Technology Corp. All rights reserved.

SigmaStar Technology makes no representations or warranties including, for example but not limited to, warranties of merchantability, fitness for a particular purpose, non-infringement of any intellectual property right or the accuracy or completeness of this document, and reserves the right to make changes without further notice to any products herein to improve reliability, function or design. No responsibility is assumed by SigmaStar Technology arising out of the application or use of any product or circuit described herein; neither does it convey any license under its patent rights, nor the rights of others.

SigmaStar is a trademark of SigmaStar Technology Corp. Other trademarks or names herein are only for identification purposes only and owned by their respective owners.

REVISION HISTORY

Revision No.	Description	Date
1.0	● Initial release	09/12/2020
1.1	● Modified API	10/15/2020
1.2	● 4MIC Beamforming and modified API	08/03/2021

TABLE OF CONTENTS

REVISION HISTORY	i
TABLE OF CONTENTS	ii
1. Overview	1
1.1. Algorithm Description	1
1.2. Keyword	1
1.2.1. Noise Gate	1
1.2.2. Diagonal loading	1
1.3. Note	1
1.3.1. Implementation	1
1.3.2. Cooperation	1
1.3.3. Correspondence	2
2. SPECIFICATION	3
3. Coordinate system of Microphone Array	4
3.1. Multichannel Microphone array	4
3.2. Multichannel Microphone array	4
3.2.1. Uniform Linear Array	4
3.2.2. Uniform Circular Array	5
4. API Reference	6
4.1. API List	6
4.2. IaaBf_GetBufferSize	6
4.3. IaaBf_Init	7
4.4. IaaBf_SetConfig	8
4.5. IaaBf_GetConfig	8
4.6. IaaBf_SetShape	9
4.7. IaaBf_Run	10
4.8. IaaBf_Reset	18
4.9. IaaBf_Free	18
5. BF Data Type	20
5.1. BF data type list	20
5.2. AudioBfInit	20

5.3. AudioBfConfig.....21

5.4. BF_HANDLE.....22

6. Error code.....24

7. FLOW.....25

1. Overview

1.1. Algorithm Description

BF (short for Beamforming) or spatial filtering is a directional signal processing technology for sensor array transmission or reception. Strengthen the signal to achieve a specific angle combination of components in the microphone array in such a way, while others will be attenuated. Beamforming can be used at the transmitter and receiver to achieve spatial selectivity.

1.2. Keyword

1.2.1. Noise Gate

Noise Gate (dBFS), below this value, the frame will be treated as noise.

If the setting is too high, it will easily cause speech distortion. If the setting is too low, it will cause Beamforming result not good enough. Please be careful to use this parameter. This parameter must be referred to the actual recording volume of microphones. If the volume is too low and this parameter is set to a large value, the frame will be treated as noise. The value range is $[-80, 0]$, the recommend value is **-20**, and the step size is 1.

1.1.1. Diagonal loading

By adding a diagonal loading to the diagonal elements of the matrix for regularization, it can be ensured that the inverse matrix must exist and hence improve the robustness of Beamforming. Please be careful to use this parameter. If the setting is too high, it will cause Beamforming result not good enough. If the setting is too low, it will cause speech distortion. The value range is $[100, 1]$, the recommend value is **10**, and the step size is 1.

1.3. Note

1.1.2. Implementation

In order to facilitate debugging and confirm the effect of the algorithm, the user application needs to implement replacement algorithm parameter and grab audio data.

1.1.3. Cooperation

With the help of Audio Process Chain, Beamforming will have better signal enhancement and interference suppression effect.

1.1.4. Correspondence

Different channel numbers will correspond to different reference libraries. The user needs to confirm whether to use the correct number of channels and the corresponding reference library.

2. SPECIFICATION

1. The number of microphone is **two or more**.
2. The microphones should be the **same type**.
3. The microphones gain should be the same.
4. For best performance, background environment should be quiet.
5. If you are using audio files as the sound source , you should make sure
 - A. There is **no signal clipping** in audio files, see Figure 1.

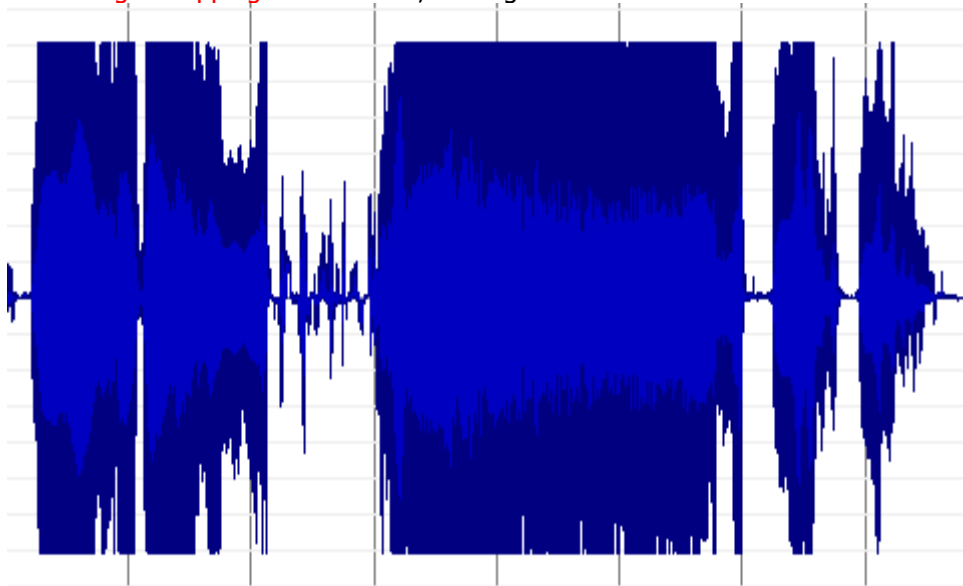


Figure 1: Audio example of clipping.

3. Coordinate system of Microphone Array

3.1. Multichannel Microphone array

Compared to the system consisting of a dual microphone array, multichannel microphone array system is capable of reaching better spatial selectivity at a specific angle because it has better bandwidth resolution and more microphone weights could be controlled. We denotes the specific angle (the angle we want to improve) microphone_doa that represents the direction of sound. There are two main types of multichannel array system used in practice, one is the uniform linear array, and the other is uniform circular array.

3.2. Multichannel Microphone array

3.2.1. Uniform Linear Array

Uniform linear array is a straight line array that is spaced evenly. Because of its DOA symmetry, we only consider the microphone_doa locates at the upper plane (from -90 degree to 90 degree). Figure 2 illustrates the uniform linear array and its coordinate system. The microphone_doa is defined as the angle between the array center and the x-axis where counterclockwise is positive. We recommend the distance of adjacent microphones should be bigger than 5cm or 6cm.

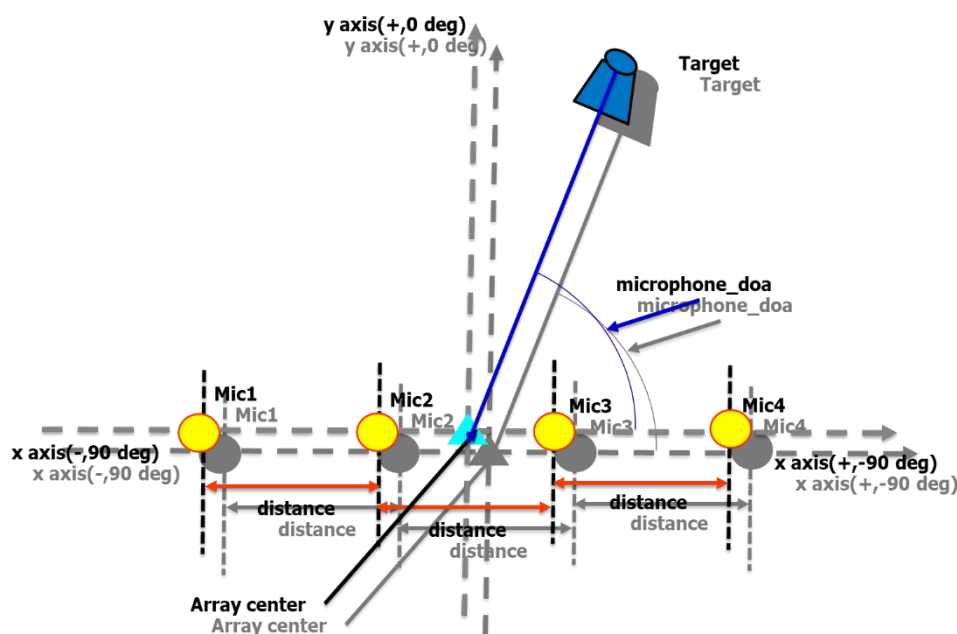


Figure 2. Uniform linear array and its coordinate system.

3.2.2. Uniform Circular Array

Uniform circular array is a circular array that the angle between adjacent microphones and array center are the same. Because of its DOA asymmetry, we consider the microphone_doa locates at the whole plane (from -90 degree to 270 degree). Figure 3 illustrates the uniform circular array and its coordinate system. The microphone_doa is defined as the angle between the array center and the x-axis where counterclockwise is positive. The distance is the diameter of the circle. We recommend the distance should be bigger than 6cm.

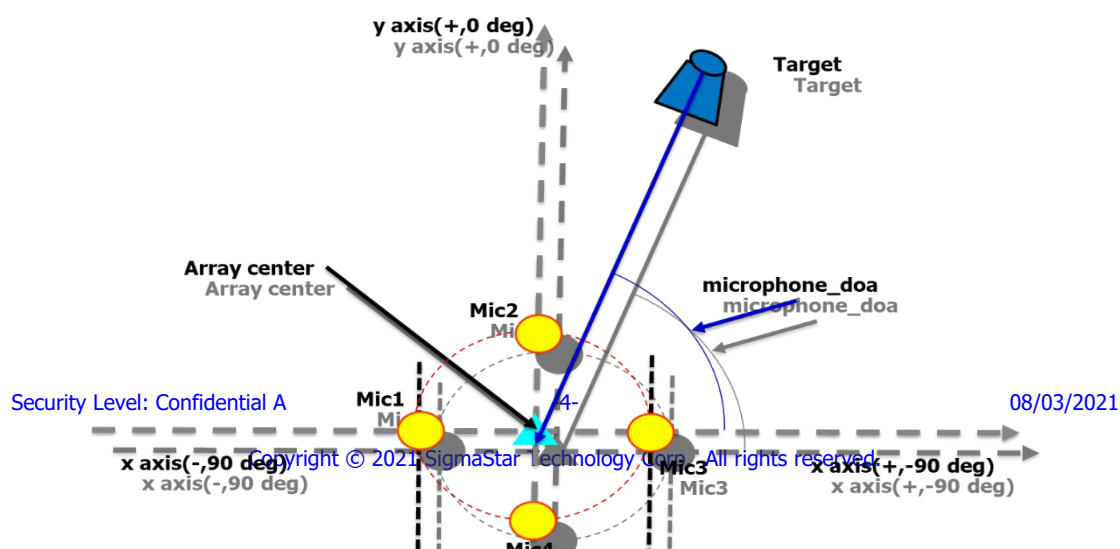


Figure 3. Uniform circular array and its coordinate system.

2. API Reference

2.1. API List

API name	Features
IaaBf_GetBufferSize	Get the memory size required for Bf algorithm running
IaaBf_Init	Initialize Bf algorithm
IaaBf_SetConfig	Configure Bf algorithm
IaaBf_GetConfig	Get the current configuration parameter information of the Bf algorithm
IaaBf_SetShape	Assign the array shape of BF.
IaaBf_Run	Bf algorithm processing
IaaBf_Reset	Reinitialize Bf algorithm
IaaBf_Free	Release Bf algorithm resources

2.2. IaaBf_GetBufferSize

➤ Features

Get the memory size required for Bf algorithm running.

➤ Syntax

```
unsigned int IaaBf_GetBufferSize(void);
```

➤ Parameters

Parameter Name	Description	Input/Output
----------------	-------------	--------------

➤ Return value

Return value is the memory size required for Bf algorithm running.

➤ Dependency

- Header: AudioBfProcess.h
- Library: libBF_2MIC_LINUX.so/ libBF_2MIC_LINUX.a/libBF_4MIC_LINUX.so/libBF_4MIC_LINUX.a

※ Note

The interface only returns the required memory size, and the application and release of memory need to be processed by the application.

➤ Example

Please refer to [IaaBf_Run](#) example.

2.3. IaaBf_Init

➤ Features

Initialize the Bf algorithm.

➤ Syntax

```
BF_HANDLE IaaBf_Init(char* working_buffer,AudioBfInit* bf_init);
```

➤ Parameters

Parameter Name	Description	Input/Output
working_buffer	Memory address used by Bf algorithm	Input
bf_init	Bf algorithm initialization structure pointer	Input

➤ Return value

Return value	Result
Not NULL	Successful
NULL	Failed

➤ Dependency

- Header: AudioBfProcess.h
- Library: libBF_2MIC_LINUX.so/ libBF_2MIC_LINUX.a/libBF_4MIC_LINUX.so/libBF_4MIC_LINUX.a

※ Note

None.

➤ Example

Please refer to [IaaBf_Run](#) example.

2.4. IaaBf_SetConfig

➤ Features

Configure Bf algorithm.

➤ Syntax

```
ALGO_BF_RET IaaBf_SetConfig(BF_HANDLE handle, AudioBfConfig* bf_config);
```

➤ Parameters

Parameter Name	Description	Input/Output
handle	Bf algorithm handle	Input
bf_config	Bf algorithm configuration structure pointer	Input

➤ Return value

Return value	Result
0	Successful
Non-zero	Failed, refer to Error code

➤ Dependency

- Header: AudioBfProcess.h
- Library: libBF_2MIC_LINUX.so/ libBF_2MIC_LINUX.a/libBF_4MIC_LINUX.so/libBF_4MIC_LINUX.a

※ Note

None.

➤ Example

Please refer to [IaaBf_Run](#) example.

2.5. IaaBf_GetConfig

➤ Features

Get the current configuration parameter information of the Bf algorithm.

➤ Syntax

```
ALGO_BF_RET IaaBf_GetConfig(BF_HANDLE handle, AudioBfConfig* bf_config);
```

➤ Parameters

Parameter Name	Description	Input/Output
handle	Bf algorithm handle	Input
bf_config	Bf algorithm configuration structure pointer	Output

➤ Return value

Return value	Result
0	Successful
Non-zero	Failed, refer to Error code

➤ Dependency

- Header: AudioBfProcess.h
- Library: libBF_2MIC_LINUX.so/ libBF_2MIC_LINUX.a/libBF_4MIC_LINUX.so/libBF_4MIC_LINUX.a

※ Note

None.

➤ Example

Please refer to [IaaBf_Run](#) example.

2.6. IaaBf_SetShape

➤ Features

Assign the array shape of BF.

➤ Syntax

```
ALGO\_BF\_RET IaaBf_SetShape(BF\_HANDLE handle,int bf_shape);
```

➤ Parameters

Parameter Name	Description	Input/Output
handle	Bf algorithm handle	Input
shape	The integer to decide array shape of BF. 0: Uniform Linear Array. 1: Uniform Circular Array.	Input

➤ Return value

Return value	Result
0	Successful
Non-zero	Failed, refer to Error code

➤ Dependency

- Header: AudioBfProcess.h
- Library: libBF_2MIC_LINUX.so/ libBF_2MIC_LINUX.a/libBF_4MIC_LINUX.so/libBF_4MIC_LINUX.a

※ Note

- Our lib only supports uniform linear array and uniform circular array (Figure 2 and Figure 3). User has to inform the modification if he/she requires special array geometry.
- Only uniform linear array exists when there are only two microphones.
- The settings of array position will impact largely on Beamforming performance. Therefore, the settings of array position must be matched to the microphone array being used.

➤ Example

Please refer to [IaaBf_Run](#) example.

2.7. IaaBf_Run

➤ Features

Bf algorithm processing.

➤ Syntax

```
ALGO_BF_RET IaaBf_Run(BF_HANDLE handle, short* microphone_input, short* microphone_output,  
float* microphone_doa);
```

➤ Parameters

Parameter Name	Description	Input/Output
handle	Bf algorithm handle	Input
microphone_input	The microphone raw data.	Input
microphone_output	Output data after Beamforming.	Output
microphone_doa	Beamforming processing angle. This value could be set by user or calculated from SSL (Need one more transformation). Set 0 for sound coming from forward, 90 for sound coming from left, and -90 for sound coming from right.	Input

➤ Return value

Return value	Result
0	Successful
Non-zero	Failed, refer to Error code

➤ Dependency

- Header: AudioBfProcess.h
- Library: libBF_2MIC_LINUX.so/ libBF_2MIC_LINUX.a/libBF_4MIC_LINUX.so/libBF_4MIC_LINUX.a

※ Note

- For dual microphone array (the microphone input is binaural data), the data pointed to microphone_input should use the sampling point as the smallest unit and be placed in the format of L,R,L,RThe length must correspond to the point_number (the number of sampling points once beamforming process) set in IaaBf_Init.
- For multichannel microphone array (microphone number is bigger than two where binaural data is not enough for processing), the input data of each microphone must be mono-channel. The data pointed to microphone_input should use the sampling point as the smallest unit and be placed in the format of [Left → Right], according to the relative position. The length must correspond to the point_number (the number of sampling points once beamforming process) set in IaaBf_Init.
- Take uniform linear array in Figure 2 as example, the microphone_input should be placed in the format of [MIC1→MIC2→MIC3→MIC4].
- The data length of microphone_output must correspond to the point_number set in IaaBf_init.
 - ◆ For 4MIC array, If point_number =128points→ microphone_input = 128x4 = 512points, and microphone_output = 128points.

➤ Example

- Example I: Dual Microphone array(The microphone input is a binaural audio)

```
1. #include <stdio.h>
2. #include <unistd.h>
3. #include <fcntl.h>
4. #include <string.h>
5. #include <sys/time.h>
6. #include <sys/ioctl.h>
7. #include <stdlib.h>
8. #include "AudioBfProcess.h"
9.
10. #define USE_MALLOC (1)
11. typedef unsigned char      uint8;
12. typedef unsigned short    uint16;
13. typedef unsigned long     uint32;
14.
15. float AVERAGE_RUN(int a)
16. {
17.     static unsigned int num = 0;
18.     static float avg = 0;
19.     if(num == 0) avg = 0;
20.     num++;
21.     avg = avg + ((float)a - avg) / ((float)num);
22.     return avg;
23. }
24. unsigned int _OsCounterGetMs(void)
25. {
26.     struct timeval t1;
27.     gettimeofday(&t1, NULL);
28.     unsigned int T = ( (1000000 * t1.tv_sec) + t1.tv_usec ) / 1000;
29.     return T;
30. }
31.
32. int main(int argc, char *argv[])
33. {
34.     short input[256];
35.     short output[128];
36.     char infileName[512];
37.     char outfileName[512];
38.
39.     FILE * fin, * fout;
40.     ALGO_BF_RET ret;
41.     int shape = 0;
42.     float direction = 0.0;
```



```
43. int delay_sample = 0;
44. float avg = 0;
45. int counter = 0;
46. unsigned int T0,T1;
47. AudioBfInit bf_init;
48. AudioBfConfig bf_config;
49. BF_HANDLE handle;
50. bf_init.mic_distance = 8.0;
51. bf_init.point_number = 128;
52. bf_init.sample_rate = 16000;
53. bf_init.channel = 2;
54. bf_config.noise_gate_dbfs = -20;
55. bf_config.temperature = 20;
56. bf_config.noise_estimation = 0;
57. bf_config.output_gain = 0.7;
58. bf_config.vad_enable = 0;
59. bf_config.diagonal_loading = 10;
60. #if USE_MALLOC
61. char *WorkingBuffer2;
62. WorkingBuffer2 = (char*)malloc(IaaBf_GetBufferSize());
63. #endif
64. handle = IaaBf_Init((char*)WorkingBuffer2, &bf_init);
65. if (handle==NULL)
66. {
67.     printf("BF init error\r\n");
68.     return -1;
69. }
70. else
71. {
72.     printf("BF init succeed\r\n");
73. }
74. ret = IaaBf_SetConfig(handle,&bf_config);
75. if (ret)
76. {
77.     printf("Error occured in Config\n");
78.     return -1;
79. }
80. ret = IaaBf_SetShape(handle,shape);
81. if (ret)
82. {
83.     printf("Error occured in Array shape\n");
84.     return -1;
85. }
```

```
86. sprintf(infileName,"%s","../sample/data/Chn-14.wav");
87. sprintf(outfileName,"%s","./BFOut.pcm");
88.
89. fin = fopen(infileName, "rb");
90. if(!fin)
91. {   printf("the input file could not be open\n");
92.     return -1;
93. }
94.
95. fout = fopen(outfileName, "wb");
96. if(!fout)
97. {
98.     printf("the output file could not be open\n");
99.     return -1;
100. }
101.
102. fread(input, sizeof(char), 44, fin); // read header 44 bytes
103. fwrite(input, sizeof(char),44, fout); // write 44 bytes output
104. // int delay_sample = 0;
105. while(fread(input, sizeof(short), bf_init.point_number*bf_init.channel, fin))
106. {
107.     counter++;
108.     T0 = (long)_OsCounterGetMs();
109.     ret = IaaBf_Run(handle,input,output,&direction);
110.     T1 = (long)_OsCounterGetMs();
111.     avg += (T1 - T0);
112.     if(ret)
113.     {
114.         printf("Error ocured in Run\n");
115.         return -1;
116.     }
117.     fwrite(output, sizeof(short), bf_init.point_number, fout);
118. }
119. avg /= counter;
120. printf("AVG is %.2f ms\n",avg);
121. IaaBf_Free(handle);
122. fclose(fin);
123. fclose(fout);
124. free(WorkingBuffer2);
125. printf("Done\n");
126. return 0;
127. }
```

- Example II: Multichannel Microphone array (Microphone input are 4x mono-channel audio)

```

1. #include <stdio.h>
2. #include <unistd.h>
3. #include <fcntl.h>
4. #include <string.h>
5. #include <sys/time.h>
6. #include <sys/ioctl.h>
7. #include <stdlib.h>
8. #include "AudioBfProcess.h"
9.
10. #define MIC_NUM (4)
11. #define USE_MALLOC (1)
12. typedef unsigned char      uint8;
13. typedef unsigned short    uint16;
14. typedef unsigned long     uint32;
15.
16. float AVERAGE_RUN(int a)
17. {
18.     static unsigned int num = 0;
19.     static float avg = 0;
20.     if(num == 0) avg = 0;
21.     num++;
22.     avg = avg + ((float)a - avg) / ((float)num);
23.     return avg;
24. }
25. unsigned int _OsCounterGetMs(void)
26. {
27.     struct timeval t1;
28.     gettimeofday(&t1, NULL);
29.     unsigned int T = ( (1000000 * t1.tv_sec) + t1.tv_usec ) / 1000;
30.     return T;
31. }
32.
33. int main(int argc, char *argv[])
34. {
35.     /*****Input file init*****/
36.     short input[512];
37.     short output[128];
38.     short input_tmp1[128], input_tmp2[128], input_tmp3[128], input_tmp4[128];
39.     FILE * fin0, * fin1, * fin2, * fin3, * fout;
40.

```

```
41. char infileName[MIC_NUM][512];
42. char outfileName[512];
43. ALGO_BF_RET ret;
44. int k;
45. float avg = 0;
46. int counter = 0;
47. unsigned int T0, T1;
48. /*****common setting between SSL and BF*****/
49. int point_number = 128;
50. float microphone_distance = 4.0;
51. int temperature = 20;
52. int sample_rate = 16000;
53. int shape = 0;
54. // int delay_sample[MIC_NUM-1] = {0,0,0}; //channel-1
55. float direction = 0.0;
56. /*****BF data init*****/
57. #if USE_MALLOC
58. char *WorkingBuffer_BF;
59. WorkingBuffer_BF = (char*)malloc(IaaBf_GetBufferSize());
60. #endif
61.
62. AudioBfInit bf_init;
63. AudioBfConfig bf_config;
64. BF_HANDLE bf_handle;
65. bf_init.mic_distance = microphone_distance;
66. bf_init.point_number = point_number;
67. bf_init.sample_rate = sample_rate;
68. bf_init.channel = MIC_NUM;
69. bf_config.noise_gate_dbfs = -20;
70. bf_config.temperature = temperature;
71. bf_config.noise_estimation = 0;
72. bf_config.output_gain = 0.7;
73. bf_config.vad_enable = 0;
74. bf_config.diagonal_loading = 10;
75. bf_handle = IaaBf_Init((char*)WorkingBuffer_BF, &bf_init);
76. if (bf_handle==NULL)
77. {
78.     printf("BF init error\r\n");
79.     return -1;
80. }
81. else
82. {
83.     printf("BF init succeed\r\n");
```

```
84. }
85. ret = IaaBf_SetConfig(bf_handle,&bf_config);
86. if (ret)
87. {
88.     printf("Error occured in Config\n");
89.     return -1;
90. }
91. ret = IaaBf_SetShape(bf_handle,shape);
92. if (ret)
93. {
94.     printf("Error occured in Array shape\n");
95.     return -1;
96. }
97. /*****open input file and output file*****/
98. sprintf(infileName[0],"%s","../sample/data/Chn-01.wav");
99. sprintf(infileName[1],"%s","../sample/data/Chn-02.wav");
100. sprintf(infileName[2],"%s","../sample/data/Chn-03.wav");
101. sprintf(infileName[3],"%s","../sample/data/Chn-04.wav");
102. sprintf(outfileName,"%s","./BFOut.pcm");
103.
104. fin0 = fopen(infileName[0], "rb");
105. if(!fin0)
106. {
107.     printf("the input file 0 could not be open\n");
108.     return -1;
109. }
110.
111. fin1 = fopen(infileName[1], "rb");
112. if(!fin1)
113. {
114.     printf("the input file 1 could not be open\n");
115.     return -1;
116. }
117. fin2 = fopen(infileName[2], "rb");
118. if(!fin2)
119. {
120.     printf("the input file 2 could not be open\n");
121.     return -1;
122. }
123. fin3 = fopen(infileName[3], "rb");
124. if(!fin3)
125. {
126.     printf("the input file 3 could not be open\n");
```

```
127.     return -1;
128. }
129. fout = fopen(outfileName, "wb");
130. if(!fout)
131. {
132.     printf("the output file could not be open\n");
133.     return -1;
134. }
135. fread(input, sizeof(char), 44, fin0);
136. fread(input, sizeof(char), 44, fin1);
137. fread(input, sizeof(char), 44, fin2);
138. fread(input, sizeof(char), 44, fin3);
139. fwrite(input, sizeof(char), 44, fout);
140. short * input_ptr;
141.
142. while(fread(input_tmp1, sizeof(short), bf_init.point_number, fin0))
143. {
144.     fread(input_tmp2, sizeof(short), bf_init.point_number, fin1);
145.     fread(input_tmp3, sizeof(short), bf_init.point_number, fin2);
146.     fread(input_tmp4, sizeof(short), bf_init.point_number, fin3);
147.     input_ptr = input;
148.     for(k=0;k<point_number;k++)
149.     {
150.         *input_ptr = input_tmp1[k];
151.         input_ptr++;
152.         *input_ptr = input_tmp2[k];
153.         input_ptr++;
154.         *input_ptr = input_tmp3[k];
155.         input_ptr++;
156.         *input_ptr = input_tmp4[k];
157.         input_ptr++;
158.     }
159.     counter++;
160.     T0 = (long)_OsCounterGetMs();
161.     ret = IaaBf_Run(bf_handle, input, output, &direction);
162.     T1 = (long)_OsCounterGetMs();
163.     avg += (T1 - T0);
164.     if(ret)
165.     {
166.         printf("Error occured in Run\n");
167.         return -1;
168.     }
169.     fwrite(output, sizeof(short), point_number, fout);
```

```

170.     }
171.     avg /= counter;
172.     printf("AVG is %.2f ms\n",avg);
173.     IaaBf_Free(bf_handle);
174.     fclose(fin0);
175.     fclose(fin1);
176.     fclose(fin2);
177.     fclose(fin3);
178.     fclose(fout);
179.     free(WorkingBuffer_BF);
180.     printf("Done\n");
181.
182.     return 0;
183. }

```

2.8. IaaBf_Reset

➤ Features

Reinitialize Bf algorithm.

➤ Syntax

```
BF_HANDLE IaaBf_Reset(BF_HANDLE working_buffer,AudioBfInit* bf_init);
```

➤ Parameters

Parameter Name	Description	Input/Output
working_buffer	Bf algorithm running memory address	Input
bf_init	Bf algorithm initialization structure pointer	Input

➤ Return value

Return value	Result
Not NULL	Successful
NULL	Failed

➤ Dependency

- Header: AudioBfProcess.h
- Library: libBF_2MIC_LINUX.so/ libBF_2MIC_LINUX.a/libBF_4MIC_LINUX.so/libBF_4MIC_LINUX.a

※ Note

None.

➤ Example

None.

2.9. IaaBf_Free

➤ Features

Release Bf algorithm resources.

➤ Syntax

```
ALGO_BF_RET IaaBf_Free(BF_HANDLE handle);
```

➤ Parameters

Parameter Name	Description	Input/Output
handle	Bf algorithm handle	Input

➤ Return value

Return value	Result
0	Successful
Non-zero	Failed, refer to Error code

➤ Dependency

- Header: AudioBfProcess.h
- Library: libBF_2MIC_LINUX.so/ libBF_2MIC_LINUX.a/libBF_4MIC_LINUX.so/libBF_4MIC_LINUX.a

※ Note

None.

➤ Example

Please refer to [IaaBf_Run](#) example.

3. BF Data Type

3.1. BF data type list

DATA TYPE	Description
AudioBfInit	Bf algorithm initialization parameter structure type
AudioBfConfig	Bf algorithm configuration parameter structure type
BF_HANDLE	Bf algorithm handle type

3.2. AudioBfInit

➤ Description

Define Bf algorithm initialization parameter structure type.

➤ Definition

```
typedef struct
{
    unsigned int point_number;
    unsigned int sample_rate;
    float mic_distance;
    unsigned int channel;
}AudioBfInit;
```

➤ Member

Member name	Description
point_number	The sampling points that Bf algorithm processed once
sample_rate	Sampling rate, currently supports 8k/16k.
mic_distance	The distance between adjacent mics, unit: cm, recommend: 5cm or 6cm
channel	Number of channels

※ Note

None.

➤ Related data types and interfaces

IaaBf_Init、IaaBf_Reset

3.3. AudioBfConfig

➤ Description

Define Bf algorithm configuration parameter structure type

➤ Definition

```
typedef struct
{
    unsigned int temperature;
    int noise_gate_dbfs;
    int noise_estimation;
    float output_gain;
    int vad_enable;
    int diagonal_loading;
}AudioBfConfig;
```

➤ Member

Member name	Description
temperature	Ambient temperature (Celsius) Celsius = (5/9) * (Fahrenheit-32) Step size is 1
noise_gate_dbfs	If the power of signal is smaller than this setting, this frame will take this frame as noise part, and use this frame to calculate some estimation. Please be careful to use this parameter. This parameter must be referred to the actual recording volume of microphones. If setting too high, it will cause the speech distortion. If setting too small, it will cause BF result not good enough. Recommend: -20.
noise_estimation	Noise estimation mode, Input: 0 or 1. 0 for adaptation method. 1 for average method. Recommend to use 0 for normal application.
output_gain	Output signal gain control (range : 0 ~ 1). 0.7 is the normal case. Setting as 1 will add 4db. Recommend: 0.7 (nothing change).

Member name	Description
vad_enable	Whether voice alive detection (VAD) mode is enable. Input: 0 or 1. If this setting is enable, VAD is used to calculate whether each frame enters the noise estimation process. If this setting is disable, the noise_gate_dbfs set above is used to determine whether it is treated as noise.
diagonal_loading	Regularization term for inverse matrix. Please be careful to use this parameter. If setting too high, it will cause BF result not good enough. If setting too low, it will cause the speech distortion. Recommend: 10.

※ Note

- None.

➤ Related data types and interfaces

[IaaBf_SetConfig](#), [IaaBf_GetConfig](#)

3.4. BF_HANDLE

➤ Description

Bf algorithm handle type.

➤ Definition

```
typedef void* BF_HANDLE;
```

➤ Member

Member name	Description
-------------	-------------

※ Note

None.

➤ Related data types and interfaces

[IaaBf_Init](#)[IaaBf_SetConfig](#)[IaaBf_GetConfig](#)[IaaBf_SetShape](#)[IaaBf_Run](#)[IaaBf_Reset](#)[IaaBf_Free](#)

4. Error code

BF API error codes are shown as follow:

Table 6-1 BF API error code

Error code	Definition	Description
0x00000000	ALGO_BF_RET_SUCCESS	BF runs successfully
0x10000301	ALGO_BF_RET_INIT_ERROR	BF initialization error
0x10000302	ALGO_BF_RET_INVALID_CONFIG	BF Config is invalid
0x10000303	ALGO_BF_RET_INVALID_HANDLE	BF Handle is invalid
0x10000304	ALGO_BF_RET_INVALID_NOISE_ESTIMATION	BF noise estimation mode setting is invalid
0x10000305	ALGO_BF_RET_INVALID_VAD_ENABLE	BF VAD mode setting is invalid
0x10000306	ALGO_BF_RET_INVALID_OUTPUT_GAIN	BF output gain setting is invalid
0x10000307	ALGO_BF_RET_INVALID_INPUT_POINTER	BF input indicator is invalid
0x100003089	ALGO_BF_RET_INVALID_SAMPLERATE	BF sampling rate is invalid
0x10000309	ALGO_BF_RET_INVALID_POINTNUMBER	BF sampling point is invalid
0x10000310	ALGO_BF_RET_INVALID_CHANNEL	BF channel number is invalid
0x10000311	ALGO_BF_RET_INVALID_CALLING	BF call API sequence error
0x10000312	ALGO_BF_RET_API_CONFLICT	Other APIs are running
0x10000313	ALGO_BF_RET_INVALID_GEOMETRY_TYPE	BF array shape is invalid.
0x10000314	ALGO_BF_RET_INVALID_MIC_DISTANCE	BF mic distance is invalid

4. FLOW

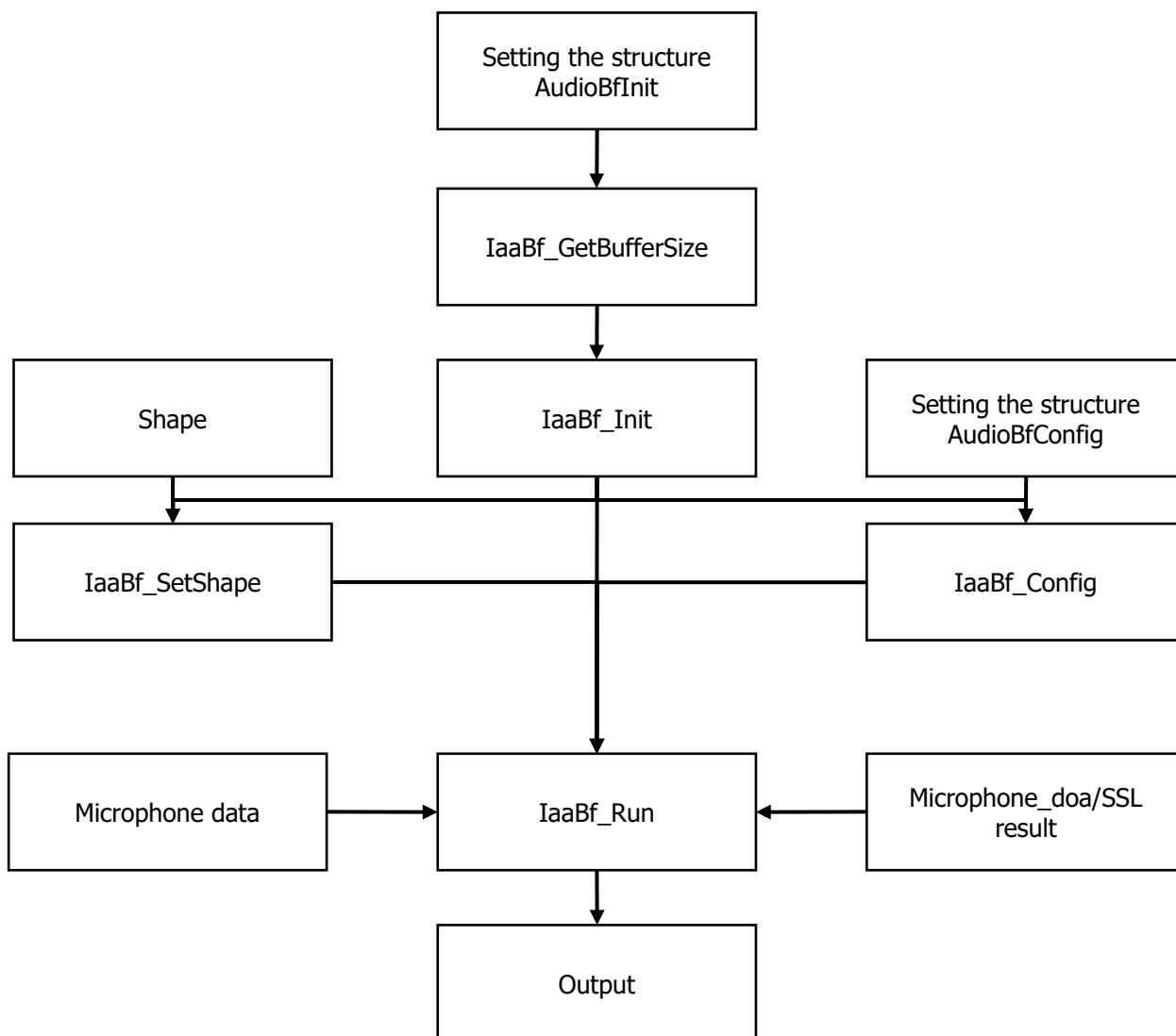


Figure 4: Beamforming working flow